

Design Rationale: The People of the Valley

1. Introduction

This document details the design for Requirement 3, with unique dialogues and behaviors. The objective is an interactive NPC system featuring a flexible, conditional monologue mechanism and distinct NPC actions (e.g., Guts's aggression), prioritizing extensibility, reusability, and object-oriented design principles (SOLID).

2. Design Goals

- Implement NPCs Sellen, Kale, and Guts with unique monologues, behaviors, and Player interaction via ListenAction.
- Support conditional monologues (e.g., Kale's rune/inventory checks, Guts's health-based dialogue) using Conditions.
- Enable Guts's AttackBehaviour (targets >50 HP, BareFist) and shared WanderBehaviour for all NPCs.

3. Design Analysis & Decisions

1. NPC Core Structure and Basic Interaction

This feature addresses the fundamental representation of NPCs (Sellen, Kale, Guts) and how the Player initiates dialogue.

Option A: Abstract Npc Class with Concrete Subclasses and ListenAction

- **Description:** An abstract `game.actors.npc.Npc` extends `engine.actors.Actor`, serving as a base for `NpcSellen`, `NpcKale`, `NpcGuts`. It manages common Behaviours (e.g., default `WanderBehaviour`) and provides `getMonologues()`. Player interaction via `ListenAction` (from `allowableActions()`) targets the NPC.
- **Maintainability:** Good. Common logic is centralized; specifics are in subclasses. New NPCs are new subclasses.

Pros	Cons
Clear Type Hierarchy (LSP); Polymorphic treatment.	Base Npc might need modification if future interactions differ vastly from <code>ListenAction</code> .

Code Reusability (DRY) for wandering and ListenAction offering.	
Encapsulation of NPC-specific data (HP, monologues).	
SRP for ListenAction (handles listening, NPC provides dialogues).	

Option B: Individual NPC Classes Extending Actor Directly, Dialogue Logic in Player

- **Description:** Each NPC extends Actor. No common Npc base. Player class checks instanceof and calls NPC-specific dialogue methods or generates actions.
- **Maintainability:** Poor. Player class modified for new NPCs (violates OCP). Common behaviours duplicated.

Pros	Cons
Superficially simpler for very few, distinct NPCs.	High Code Duplication (DRY violation).
	Poor Extensibility (OCP violation in Player).
	Tight Coupling of Player to all concrete NPC types.
	LSP violation; difficult uniform NPC treatment without instanceof.

Decision & Justification

- **Chosen Design: Option A.**
- **Rationale:** Option A aligns with OOP principles. The abstract Npc class provides a solid foundation for common functionalities (wandering, ListenAction) and polymorphism. It promotes code reuse (DRY) and encapsulation (LSP, SRP), making the system

organized, maintainable, and extensible compared to the tightly coupled Option B.

3.2. Monologue System and Conditional Dialogue Delivery

This feature details how NPCs manage and deliver dialogues, especially conditional ones.

Option A: Monologue Class with Associated Condition Interface

- **Description:** `game.actors.npc.Monologue` holds dialogue text and a `game.conditions.Condition` instance. Condition interface (`check()` method) determines availability. Concrete conditions (`LowHealthCondition`, `LowRunesCondition`, `EmptyInventoryCondition`, `NearbyCapabilityCondition`, `DefaultCondition`) implement this. NPCs override `getMonologues()` to provide a list of Monologue objects with appropriate Conditions. `ListenAction` calls `getMonologues()`, filters by `Condition.check()`, and randomly selects one.
- **Maintainability:** Excellent. New dialogues are new Monologue objects. New triggers are new Condition implementations, without altering core `Npc` or `ListenAction`.

Pros	Cons
High Cohesion (SRP): Monologue for text/trigger; Condition for specific check.	May lead to many small Condition classes if not designed for reusable parameterization.
Excellent Extensibility (OCP): New conditions/monologues added without modifying core classes.	
Decoupling (DIP): <code>Npc/ListenAction</code> depend on Condition abstraction.	
Reusability: Condition classes can be parameterized.	

Option B: Hardcoded Conditional Logic in NPC Methods

- **Description:** Dialogue availability logic (e.g., if (farmer.getRunes() < 500)) embedded directly in each NPC's getMonologues() method using if-else if.
- **Maintainability:** Poor. Methods become complex (violates SRP). Shared condition changes require editing multiple classes.

Pros	Cons
Might seem initially simpler for few NPCs with simple conditions.	Violates OCP: New conditions require modifying NPC methods.
	Low Cohesion: NPC classes handle core identity and complex conditional logic.
	Poor Reusability: Logic tightly coupled to NPC methods.
	Difficult to Test: Complex conditional blocks are harder to unit test.

Decision & Justification

- **Chosen Design: Option A.**
- **Rationale:** Option A is superior for modularity, extensibility, and adherence to OOP (SRP, OCP). The Strategy Pattern (via Condition interface) cleanly separates condition-checking from Monologue data and Npc classes, making updates easier and fostering a robust, maintainable design.

3.3. Guts's Aggressive Behavior

This feature covers Guts's combat AI (attacking actors with >50 HP).

Option A: Dedicated AttackBehaviour Class

- **Description:** game.behaviours.AttackBehaviour implements engine.actors.Behaviour. An instance (with health hurdle 50) is added to NpcGuts's behaviours (high priority). AttackBehaviour.getAction() scans surroundings, checks target HP

(>50 via BaseActorAttributes.HEALTH), and returns an AttackAction(target, direction, actor.getIntrinsicWeapon()). NpcGuts uses BareFist.

- **Maintainability:** Good. Guts's attack logic is encapsulated. Changes are localized to this behaviour.

Pros	Cons
SRP: AttackBehaviour solely decides when/whom Guts attacks.	Current AttackBehaviour is specific to "> X HP" rule. Other NPCs might need generalization or new variants for different rules.
Modularity: Separates combat AI from other NPC concerns.	
Clear AI Structure: Fits engine's behaviour system.	

Option B: Attack Logic Directly in NpcGuts.playTurn()

- **Description:** All scanning, HP checking, and AttackAction creation embedded in NpcGuts.playTurn().
- **Maintainability:** Poor. playTurn() becomes overly complex (violates SRP), mixing movement and combat AI.

Pros	Cons
Might seem slightly more straightforward for a simple attack rule.	Violates SRP: playTurn() has too many concerns.
	Low Cohesion: Attack logic intertwined with other AI logic.
	Poor Reusability: Attack logic difficult to reuse.
	Harder to Test: Complex playTurn() methods are challenging to unit

	test.
--	-------

Decision & Justification

- **Chosen Design: Option A.**
- **Rationale:** Option A aligns with the behaviour-driven architecture. Encapsulating aggression in AttackBehaviour promotes SRP, cleaner NpcGuts class, and simpler playTurn(). It's more testable and modifiable. Future extensibility (e.g., parameterizing AttackBehaviour with a Condition for varied attack triggers) is feasible.

4. Conclusion

The design for REQ3, "The People of the Valley," effectively integrates interactive NPCs by leveraging an abstract Npc base class, a flexible Monologue system with Condition-based triggers, and distinct Behaviour classes (WanderBehaviour, AttackBehaviour). This structure fulfills all specified functionalities for Sellen, Kale, and Guts, including their unique dialogues and Guts's targeted aggression. Adherence to SOLID principles (SRP, OCP, DIP) ensures a maintainable, extensible, and robust system, paving the way for future enhancements with new characters and more complex interactions.